Introduction to Lambda Calculus

Zoe Paraskevopoulou

School of Electrical and Computer Engineering
National Technical University of Athens

2025



Lambda Calculus

- Abstract model of computation (Church, 1920s)
- Function definitions, function applications, variables
- Foundation for many programming language features
  - **A Correspondence between ALGOL 60 and Church's Lambda-notation**, Landin, 1965
  - **The Next 700 Programming Languages**, Landin, 1965
- Can be extended with various syntactic constructs, leading to languages similar to ML

Syntax (Concrete)

$$t, u ::= x \mid t\ u \mid \lambda x.\, t \mid (t)$$

Syntax (Abstract)

$$t, u ::= x \mid t\ u \mid \lambda x.\, t$$

Syntax (Abstract)

$$t, u ::= x \mid t\ u \mid \lambda x.\, t$$

x

Variable

$t\ u$

Application
(*function application*)

$\lambda x.\, t$

Lambda abstraction
Anonymous function
(`fun x -> t`)

Examples

$$\lambda x.\, x$$

$$\lambda x.\, \lambda y.\, y$$

$$\lambda f.\, \lambda x.\, f\ x$$

$$(\lambda x.\, x)\ (\lambda f.\, \lambda x.\, f\ x)$$

Precedence and Associativity

- **Function application** is *left-associative*

$$t\ u\ v\ =\ (t\ u)\ v$$

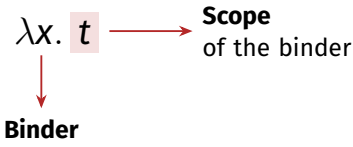
- **Function application** has *higher precedence* than lambda abstraction

$$\lambda x.\, x\, y\ =\ \lambda x.\, (x\, y)$$

- **Notation:** we write $\lambda f\, x.\, f\ x$ instead of $\lambda f.\, \lambda x.\, f\ x$

Bound Variables

Variables that are in the scope of a lambda are called *bound*.



E.g.

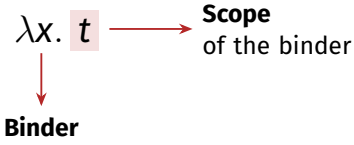
$\lambda x.\, x$

$\lambda f.\, \lambda x.\, f\ x$

$\lambda x.\, \lambda x.\, x$

Bound Variables

Variables that are in the scope of a lambda are called *bound*.



E.g.

$\lambda x.\, x$

$\lambda f.\, \lambda x.\, f\ x$

$\lambda x.\, \lambda x.\, x$

Free Variables

Variables that are not bound by any binder.

$$\lambda x.\ \lambda y.\ x\ (y\ z)$$

z is a **free variable**

Semantics

- We need to define what it means to *apply* a function
- E.g. intuitively the term

$$(\lambda f\, x.\, f\ x)\ (\lambda y.\, y)\ (\lambda y.\, y)$$

evaluates to

$$\boldsymbol{\lambda y.\, y}$$

- How?

Substitution

It is useful to define the **substitution** of a **variable** x that appears **free** in a term t, with another term u.

We define it by structural recursion on terms.

$$
\begin{aligned}
[x \to u]\, x &= \\
[x \to u]\, y &= \\
[x \to u]\, t_1\, t_2 &= \\
[x \to u]\, \lambda x.\, t &= \\
[x \to u]\, \lambda y.\, t &=
\end{aligned}
$$

$$[x \to u]\, t$$

Substitution

It is useful to define the **substitution** of a **variable** x that appears **free** in a term t, with another term u.

$$[x \to u]\, t$$

We define it by structural recursion on terms.

$$
\begin{array}{rcll}
[x \to u]\, x & = & u & \\
[x \to u]\, y & = & y & \text{if } x \neq y \\
[x \to u]\, t_1\ t_2 & = & [x \to u]\, t_1\ \ [x \to u]\, t_2 & \\
[x \to u]\, \lambda x.\, t & = & \lambda x.\, t & \\
[x \to u]\, \lambda y.\, t & = & \lambda y.\, [x \to u]\, t & \text{if } x \neq y \text{ and } y \notin \mathbf{FV}(u)
\end{array}
$$

ensures capture avoidance

Substitution: Examples

$$[z \to \lambda y.\, y]\,(\lambda f\, x.\, f\ x\ z) =$$

$$[y \to \lambda z.\, z]\,(\lambda f.\, f\ (\lambda y.\, y)\ y) =$$

Substitution: Examples

$$[z \to \lambda y.\, y]\,(\lambda f\, x.\, f\ x\ z) = \lambda f\, x.\, f\ x\ (\lambda y.\, y)$$

$$[y \to \lambda z.\, z]\,(\lambda f.\, f\ (\lambda y.\, y)\ y) = \lambda f.\, f\ (\lambda y.\, y)\ (\lambda z.\, z)$$

Capture Avoiding Substitution

When substituting a term into a function, the free variables of the term must not be captured by the function's binder. That would change the meaning.

$$[x \to z]\,(\lambda f\, z.\, f\, x\, z) \neq \lambda f\, z.\, f\, z\, z$$

Instead: **Fresh name!** (*alpha conversion*)

$$[x \to z]\,(\lambda f\, z.\, f\, x\, z) = \lambda f\, z'.\, f\, z\, z'$$

Alpha Equivalence

Two terms that differ only in the names of their bound variables (equal up to a **consistent renaming** of their bound variables).

E.g.

$$\lambda f\, z.\, f\ z\ y \ =_\alpha \ \lambda g\, x.\, g\ x\ y$$

But

$$\lambda f\, z.\, f\ z\ y \ \neq_\alpha \ \lambda x\, x.\, x\ x\ y \qquad\qquad \lambda f\, z.\, f\ z\ y \ \neq_\alpha \ \lambda f\, y.\, f\ y\ y$$

Capture Avoiding Substitution

- To avoid the problem, when working on paper we simply ignore it…
  - Working implicitly up to alpha conversion
- In a proof assistant it is not so easy…
  - Several techniques (named binders, nameless representations (DeBruijn indices), higher-order abstract syntax)
  - POPLmark challenge

Small-Step Semantics

$$\frac{}{(\lambda x.\, t_1)\; t_2 \to [x \to t_2]\, t_1}\ \textsc{App}_1$$

Small-Step Semantics

$$\frac{}{(\lambda x.\, t_1)\; t_2 \to [x \to t_2]\, t_1}\ \textsc{App}_1$$

$(\lambda x.\, t_1)\; t_2$ is a β**-redex**

This rule is called β**-reduction**

Small-Step Semantics

Full beta-reduction

Any redex can be reduced at any time.

$$\frac{}{(\lambda x.\, t_1)\; t_2 \to [x \to t_2]\, t_1}\ \textsc{App}_1 \qquad \frac{t_1 \to t_1'}{t_1\; t_2 \to t_1'\; t_2}\ \textsc{App}_2$$

$$\frac{t_2 \to t_2'}{t_1\; t_2 \to t_1\; t_2'}\ \textsc{App}_3 \qquad \frac{t \to t'}{\lambda x.\, t \to \lambda x.\, t'}\ \textsc{Lam}$$

Example

$$
\begin{aligned}
(\lambda x.\, y)\,\underline{((\lambda z.\, z\, z)(\lambda t.\, t))} &\longrightarrow_\beta (\lambda x.\, y)\,\underline{((\lambda t.\, t)(\lambda t.\, t))} \\
&\longrightarrow_\beta \underline{(\lambda x.\, y)\,(\lambda t.\, t)} \\
&\longrightarrow_\beta y
\end{aligned}
$$

Normal form

A term that cannot be reduced further

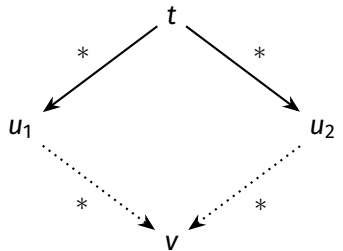
Example

$$\Omega = (\lambda x.\, x\, x)(\lambda x.\, x\, x)$$

$$(\lambda x.\, x\, x)(\lambda x.\, x\, x) \longrightarrow_\beta (\lambda x.\, x\, x)(\lambda x.\, x\, x) \longrightarrow_\beta (\lambda x.\, x\, x)(\lambda x.\, x\, x) \longrightarrow_\beta \cdots$$

Confluence (Church–Rosser Theorem)

$$\lambda y.\, y \xleftarrow{\beta} (\lambda x\, y.\, y)((\lambda x.\, x)(\lambda x.\, x)) \xrightarrow{\beta} (\lambda x\, y.\, y)(\lambda x.\, x)$$



Corollary: Normal forms are unique.

Evaluation Strategies

- **Normal order:** leftmost, outermost redex is reduced first
- **Applicative order:** leftmost, innermost redex is reduced first
- **CBV:** like applicative, but **no reduction under lambdas**
- **CBN:** like normal, but **no reduction under lambdas**

Call-by-value call-by-name and the lambda calculus. G. B. Plotkin

Values

When is an expression considered fully evaluated?

$$\text{value}(\lambda x.\, t) = \text{true}$$

$$\text{value}(_) = \text{false}$$

Small-Step Semantics, Call By Value

$$\frac{\text{value}(v_2) = \text{true}}{(\lambda x.\, t_1)\ v_2 \to [x \to v_2]\, t_1}\ \textsc{App}_1 \qquad \frac{t_1 \to t_1'}{t_1\ t_2 \to t_1'\ t_2}\ \textsc{App}_2 \qquad \frac{\text{value}(v_1) = \text{true} \qquad t_2 \to t_2'}{v_1\ t_2 \to v_1\ t_2'}\ \textsc{App}_3$$

Innermost redex first

Strict strategy

Leftmost redexes first

Left-to-right evaluation order

Small-Step Semantics, Call By Name

$$\frac{}{(\lambda x.\, t_1)\; t_2 \to [x \to t_2]\, t_1}\ \textsc{App}_1 \qquad \frac{t_1 \to t_1'}{t_1\; t_2 \to t_1'\; t_2}\ \textsc{App}_2$$

Outermost redexes first

Strict strategy

Leftmost redexes first

Left-to-right evaluation order

Arguments are not evaluated

Evaluation only when needed

(here the only values are functions)

Turing Completeness

- We can encode exactly the recursive functions
- We can encode a Turing Machine!
- **Undecidable** problems:
  - Are two lambda terms **equivalent**?
  - Is a term **normalizable**?

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Successor?

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Successor

$\text{succ} = \lambda n\, f\, x.\, f\, (n\, f\, x)$

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Successor

$\mathrm{succ} = \lambda n\, f\, x.\, f\, (n\, f\, x)$

Addition?

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Successor
$\mathsf{succ} = \lambda n\, f\, x.\, f\, (n\, f\, x)$

Addition
$\mathsf{add} = \lambda m\, n.\, m\ \mathsf{succ}\ n$

Multiplication
$\mathsf{mul} = \lambda m\, n\, f\, x.\, m\ (\mathsf{add}\ n)\ \underline{0}$

Exponentiation
$\mathsf{exp} = \lambda m\, n.\, n\ (\mathsf{mul}\ m)\ \underline{1}$

Numbers (Church Numerals)

$$\underline{0} = \lambda f\, x.\, x \qquad \underline{1} = \lambda f\, x.\, f\, x \qquad \underline{2} = \lambda f\, x.\, f(f\, x) \qquad \underline{3} = \lambda f\, x.\, f(f(f\, x)) \qquad \cdots$$

Successor

$\mathsf{succ} = \lambda n\, f\, x.\, f\, (n\, f\, x)$

Addition

$\mathsf{add} = \lambda m\, n.\, m\ \mathsf{succ}\ n$

Multiplication

$\mathsf{mul} = \lambda m\, n\, f\, x.\, m\ (\mathsf{add}\ n)\ \underline{0}$

Exponentiation

$\mathsf{exp} = \lambda m\, n.\, n\ (\mathsf{mul}\ m)\ \underline{1}$

Predecessor

$\mathsf{pred} = \lambda n\, f\, x.\, n\, (\lambda g\, h.\, h\, (g\, f))\, (\lambda y.\, x)\, (\lambda y.\, y)$

Subtraction

$\mathsf{sub} = \lambda m\, n.\, n\ \mathsf{pred}\ m$

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad\qquad \mathsf{F} = \lambda x\, y.\, y$$

If?

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad\qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Boolean connectives?

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Boolean connectives

$\mathsf{and} = \lambda x\, y.\, x\ y\ \mathsf{F}$ $\qquad$ $\mathsf{or} = \lambda x\, y.\, x\ \mathsf{T}\ y$

$\mathsf{not} = \lambda x.\, x\ \mathsf{F}\ \mathsf{T}$

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Boolean connectives

$\mathsf{and} = \lambda x\, y.\, x\ y\ \mathsf{F}$ $\qquad \mathsf{or} = \lambda x\, y.\, x\ \mathsf{T}\ y$

$\mathsf{not} = \lambda x.\, x\ \mathsf{F}\ \mathsf{T}$

What happens in CBV?

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Boolean connectives

$\mathsf{and} = \lambda x\, y.\, x\, y\, \mathsf{F}$ $\qquad$ $\mathsf{or} = \lambda x\, y.\, x\, \mathsf{T}\, y$

$\mathsf{not} = \lambda x.\, x\, \mathsf{F}\, \mathsf{T}$

Is zero?

Booleans

$$\mathsf{T} = \lambda x\, y.\, x \qquad \mathsf{F} = \lambda x\, y.\, y$$

If

$\mathsf{if} = \lambda b\, x\, y.\, b\, x\, y$

Boolean connectives

$\mathsf{and} = \lambda x\, y.\, x\ y\ \mathsf{F}$ $\qquad$ $\mathsf{or} = \lambda x\, y.\, x\ \mathsf{T}\ y$

$\mathsf{not} = \lambda x.\, x\ \mathsf{F}\ \mathsf{T}$

Is zero?

$\mathsf{iszero} = \lambda n\, x\, y.\, n\, (\lambda z.\, y)\, x$

Also: pairs, lists…

Recursion

- How can we compute recursive functions?
- Let

$$\text{fact}' = \lambda \mathit{fact}.\, \lambda n.\ \text{if}\ (\text{iszero}\ n)\ \underline{1}\ (\text{mul}\ n\ (\mathit{fact}\ (n - \underline{1})))$$

- In mathematics, the *fixed point* of a function f is a value such that $f(x) = x$
- Why? Let fact be the fixed point of fact′. Then fact $=$ fact′ fact. Therefore:

$$\text{fact} = \lambda n.\ \text{if}\ (\text{iszero}\ n)\ \underline{1}\ (\text{mul}\ n\ (\text{fact}\ (n - \underline{1})))$$

- The recursive function that computes the factorial is the fixed point of fact′!

Fixed Points

- In the lambda calculus, every term has a fixed point
- We want a term Y such that for every t:

$$\mathsf{Y}\ t \longrightarrow_\beta t\ (\mathsf{Y}\ t)$$

- So for every term t, $(\mathsf{Y}\ t)$ is a fixed point
- Y is called a **fixed point combinator**

Y Combinator

- Y combinator (CBN)

  $\mathsf{Y} = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$

- Z combinator (CBV) (or *strict* Y combinator)

  $\mathsf{Z} = \lambda f.\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y))$

- E.g.

  $\mathsf{fact} = \mathsf{Z}\ (\lambda \mathit{fact}.\ \lambda n.\ \mathsf{if}\ (\mathsf{iszero}\ n)\ \underline{1}\ (\mathsf{mul}\ n\ (\mathit{fact}\ (n - \underline{1}))))$

Y Combinator

- Y combinator (CBN)

  $\mathsf{Y} = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$

- Z combinator (CBV) (or *strict* Y combinator)

  $\mathsf{Z} = \lambda f.\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y))$

- E.g.

  $\mathsf{fact} = \mathsf{Z}\ (\lambda \mathit{fact}.\ \lambda n.\ \mathsf{if}\ (\mathsf{iszero}\ n)\ \underline{1}\ (\mathsf{mul}\ n\ (\mathit{fact}\ (n - \underline{1}))))$

$$
\begin{aligned}
\mathsf{Y}\ t &\equiv (\lambda f.\ (\lambda x.\ f\ (x\,x))\ (\lambda x.\ f\ (x\,x)))\ t \\
&\longrightarrow_\beta (\lambda x.\ t\ (x\,x))\ (\lambda x.\ t\ (x\,x)) \\
&\longrightarrow_\beta t\ ((\lambda x.\ t\ (x\,x))\ (\lambda x.\ t\ (x\,x))) \\
&\longleftarrow_\beta t\ (\mathsf{Y}\ t)
\end{aligned}
$$

Y Combinator – Example

where $F = \lambda f\, n.\, \text{if } (\text{iszero } n)\ \underline{1}\ (\text{mul } n\ (f\ (\text{pred } n)))$

$$
\begin{aligned}
\text{fact } \underline{2} &\equiv (\mathsf{Y}\, F)\ \underline{2} \\
&=_\beta F\ (\mathsf{Y}\, F)\ \underline{2} \\
&\longrightarrow_\beta^* \text{if } (\text{iszero } \underline{2})\ \underline{1}\ (\text{mul } \underline{2}\ ((\mathsf{Y}\, F)\ (\text{pred } \underline{2}))) \\
&\longrightarrow_\beta^* \text{if false } \underline{1}\ (\text{mul } \underline{2}\ ((\mathsf{Y}\, F)\ (\text{pred } \underline{2}))) \\
&\longrightarrow_\beta^* \text{mul } \underline{2}\ ((\mathsf{Y}\, F)\ (\text{pred } \underline{2})) \\
&\longrightarrow_\beta^* \text{mul } \underline{2}\ ((\mathsf{Y}\, F)\ \underline{1}) \\
&\ \ \vdots \\
&\longrightarrow_\beta^* \text{mul } \underline{2}\ (\text{mul } \underline{1}\ \underline{1}) \\
&\longrightarrow_\beta^* \underline{2}
\end{aligned}
$$

References

- Proof = Program. Samuel Mimram.